World Cup 2026 Prediction Engine

Technical Reference & Concept Guide

Version: 1.0

Project: World Cup Simulator (University of Toronto)

Audience: Students, mentors, and reviewers (including S&T / quant-focused readers)

Table of Contents

---

1. Executive Summary

This project is a probabilistic forecasting engine for the 2026 FIFA World Cup. It does not output a single predicted winner; it outputs probability distributions over tournament outcomes (who wins, who reaches each knockout round, who advances from groups).

The engine combines:

- Historical international match data (goals, dates, competitions)
- A Dixon-Coles bivariate Poisson model (industry-standard for football scorelines)
- Elo rating priors (anchor teams with sparse data)
- Tournament-importance weighting (World Cup matches count more than weak qualifiers)
- Vectorized Monte Carlo simulation (100,000 full tournaments in seconds)
- Calibration and market benchmarking (Brier score, reliability diagrams, bookmaker odds)

The codebase is modular: each stage reads and writes clean interfaces (pandas DataFrames and parameter dictionaries), so you can test and debug sections independently.

---

2. What This Engine Produces

Output	Description
Team attack/defense parameters	How strong each nation is at scoring and preventing goals (log-scale)
Match probabilities	P(home win), P(draw), P(away win) for any pairing
Group advancement %	Probability each team escapes the group stage
Knockout round probabilities	R16, QF, SF, finalist, and champion frequencies
win_probs.json	Saved JSON with all probability tables
Reliability diagram	PNG chart for calibration backtests
Market edges (optional)	Where model probability differs from bookmaker implied probability

---

3. High-Level Architecture

```
[results.csv] [elo_ratings.csv] | | v v +---------+ +-----------+ | DATA | | FEATURES | | fetch | | ratings | | clean | | (Elo join)| +----+----+ +-----+-----+ | | +--------+-----------+ v +-------------+ | MODEL | | Dixon-Coles | | (fit MLE) | +------+------+ | v +-------------+ | SIMULATION | | Monte Carlo | | 2026 bracket| +------+------+ | +--------+--------+ v v +---------+ +-----------+ |evaluate | | app.py | |calibrate| | Streamlit | | market | | dashboard | +---------+ +-----------+
```

Design principle: config.py holds every hyperparameter. No magic numbers inside model files.

---

4. End-to-End Pipeline

Step 1 - Load & clean data

- Read Mart Jurisoo international results CSV
- Keep competitive matches since 2018-01-01
- Standardize team names (USA, South Korea, etc.)
- Apply exponential time decay: weight = exp(-decay_rate * days_ago)
- Multiply by tournament importance weights

Step 2 - Attach Elo (optional)

- Load elo_ratings.csv from eloratings.net
- Join home/away Elo at match date
- Convert latest Elo snapshot into attack/defense priors for the fitter

Step 3 - Fit Dixon-Coles

- Maximize weighted log-likelihood (L-BFGS-B optimizer)
- Learn per-team attack, defense, global home advantage, rho correction
- Output: DixonColesParams object

Step 4 - Monte Carlo tournament

- Simulate all group matches (vectorized across N simulations)
- Rank groups; select top 2 + 8 best third-place teams
- Simulate R32 → R16 → QF → SF → Final with extra time & penalties
- Count champion frequency → win probability

Step 5 - Evaluate (optional)

- Backtest on held-out matches (e.g. post-2022-11-01)
- Brier score, log-loss, reliability diagram
- Compare to bookmaker odds via The Odds API

---

5. Glossary of Key Terms

Term	Definition
Attack parameter	Log-scale scoring strength of a team in the Dixon-Coles model. Higher = more expected goals.
Defense parameter	Log-scale ability to suppress opponent goals. Lower (more negative) = stronger defense.
Lambda (lambda)	Expected goals for a team in a match: lambda = exp(attack + opp_defense + home_adv).
Poisson distribution	Models count data (goals). Mean = variance = lambda.
Dixon-Coles model	Bivariate Poisson for home/away goals with correlation correction for low scores (0-0, 1-0, 0-1, 1-1).
Rho (rho)	Dixon-Coles low-score correlation parameter. Typically small negative value.

Term	Definition
Home advantage	Extra log-goals added to the home team when venue is not neutral.
Neutral venue	No home advantage applied (most World Cup knockouts; non-host group games).
MLE	Maximum likelihood estimation - find parameters that best explain observed scores.
Negative log-likelihood (NLL)	Loss function minimized by the optimizer. Lower = better fit.
L2 ridge / regularization	Penalty keeping parameters from exploding; stabilizes sparse teams.
Elo prior	Soft constraint pulling attack/defense toward Elo-implied strength.
Time decay	Older matches receive lower weight in the likelihood.
Tournament weight	Multiplier by competition type (World Cup > qualifiers).
Monte Carlo simulation	Repeat random tournament many times; frequencies estimate probabilities.
Brier score	Mean squared error of probabilistic predictions. Lower is better. 0.25 = coin flip.
Log-loss	Penalizes confident wrong predictions heavily.
Reliability diagram	Plot predicted probability bins vs actual frequency. Perfect = diagonal line.
Implied probability	1 / decimal_odds, normalized to remove bookmaker margin (vig).
Vig (overround)	Bookmaker margin; raw implied probs sum to > 100%.
Edge	model_prob - market_prob. Positive = model more bullish than market.
Kelly criterion	Optimal bet fraction given edge and odds: (b*p - q) / b.
Fractional Kelly	Kelly divided by 4 (or similar) for conservative sizing.
Vectorization	NumPy array operations across all simulations at once (speed).
Third-place qualifier	8 of 12 third-placed group teams advance in 2026 format.
Extra time	Knockout ties after 90 min: lambdas scaled by 1/3 for 30 minutes.

Term	Definition
Penalty shootout	If still tied; uses historical team shootout win rates.

---

6. Statistical & Quantitative Concepts

6.1 Why Poisson goals?

Goals in football are count data. The Poisson distribution is the standard starting point: one parameter lambda controls both mean and variance. Independence between home and away goals is assumed before the Dixon-Coles correction.

6.2 Dixon-Coles correction

Pure Poisson underpredicts 0-0 draws and mis-fits 1-1 results at international level. Dixon & Coles (1997) multiply the joint probability by tau(i,j) for scores where both teams score ≤1 goal:

- (0,0): tau = 1 - lambda_h * lambda_a * rho
- (0,1): tau = 1 + lambda_h * rho
- (1,0): tau = 1 + lambda_a * rho
- (1,1): tau = 1 - rho
- Otherwise: tau = 1

6.3 Expected goals (lambdas)

For home team H vs away team A:

```
lambda_home = exp(attack_H + defense_A + home_advantage) [if not neutral] lambda_away = exp(attack_A
+ defense_H)
```

Parameters are on log scale so they stay positive after exponentiation.

6.4 Weighted likelihood

Each historical match contributes:

```
weight = exp(-decay * days_ago) * tournament_weight log L += weight * [ log Poisson(hg|lambda_h) +
log Poisson(ag|lambda_a) + log tau ]
```

6.5 Elo priors

Teams with few matches can overfit. Elo from eloratings.net provides a prior mean for attack/defense. The optimizer penalizes deviation:

```
penalty = strength * sum((attack - prior_attack)² + (defense - prior_defense)²)
```

6.6 Monte Carlo for tournaments

Analytic bracket probability is intractable for 48 teams. Simulating 100,000 tournaments:

```
P(Argentina wins) ≈ (# sims where Argentina champion) / 100,000
```

Law of large numbers: more sims → smoother estimates.

---

7. Section 1: Data Pipeline

data/fetch.py

- load_raw_data(filepath) - Reads Mart Jurisoo CSV. Requires columns: date, home_team, away_team, home_score, away_score, tournament, neutral.
- fetch_football_data_org() - Optional API top-up from football-data.org.
- generate_synthetic_results() - Fake data for smoke tests without CSV.

data/clean.py

- filter_competitive() - Drops friendlies; keeps World Cup, Euros, Copa, AFCON, qualifiers, etc.
- filter_min_date() - Default: matches on/after 2018-01-01.
- standardize_names() - Applies NAME_MAP from config (e.g. United States → USA).
- apply_decay() - Adds days_ago and weight = exp(-decay_rate * days_ago).
- apply_tournament_weights() - Multiplies weight by TOURNAMENT_WEIGHTS (World Cup ×4, AFC qualifiers ×0.4, etc.).
- build_clean_dataset() - End-to-end entry point for the pipeline.

data/fetch_elo.py

- Downloads Elo history from eloratings.net TSV files (not HTML scraping).
- parse_team_history_tsv() - Correctly reads home Elo (column 10) or away Elo (column 11) per match.
- reconcile_with_world_snapshot() - Overwrites latest rating with authoritative World.tsv.
- build_elo_csv() - CLI: python -m world_cup_model.data.fetch_elo

---

8. Section 2: Feature Engineering (Elo)

features/ratings.py

- load_elo_data() - CSV with columns: team, date, elo.

- get_elo_at_date() - Most recent Elo on or before match date (default 1500).
- build_feature_matrix() - Adds home_elo, away_elo, elo_diff via pandas merge_asof.
- current_team_elos() - Latest Elo snapshot per team for priors.
- elo_to_prior() - Maps Elo gap to prior attack/defense means (scale: ~600 Elo points per log-goal unit).

Elo is used in fitting (priors) and optionally as diagnostic columns. It is not a separate ML model layer.

---

9. Section 3: Dixon-Coles Model

model/dixon_coles.py

Key objects:

- DixonColesParams - dataclass holding attack, defense, home_adv, rho, convergence flag.

Key functions:

- _poisson_logpmf() - Stable log-Poisson via scipy.special.gammaln.
- _dc_correction_vec() - Vectorized tau correction.
- fit_model() - L-BFGS-B optimization with bounds, Elo priors, neutral-venue handling.
- predict_lambdas() - Single match expected goals.
- predict_lambdas_batch() - Vectorized for simulation (fast).
- predict_match_probabilities() - Full score grid up to 10 goals; returns p_home, p_draw, p_away.

Identifiability: Attack/defense are only identified up to constant; Elo priors + L2 + bounds prevent drift.

---

10. Section 4: Monte Carlo Tournament Simulation

simulation/tournament.py

2026 format implemented:

- 12 groups × 4 teams, round-robin (6 matches per group)
- Top 2 per group + 8 best third-place teams → 32-team knockout
- R32 → R16 → QF → SF → Final
- Hosts (USA, Canada, Mexico): home advantage in group stage only

Key functions:

- _simulate_group_stage() - All group Poisson draws shape (n_sims, n_matches).
- _rank_within_group() - Points → GD → GF → random jitter tiebreak.

- _rank_third_place_teams() - Picks best 8 of 12 third-place teams.
- _simulate_knockout_match() - 90 min, extra time (lambda × 1/3), penalties.
- run_tournament() - Public entry; returns win_probs, finalist_probs, semi_probs, etc.

Performance: 100,000 simulations in ~5-10 seconds via NumPy broadcasting.

---

11. Section 5: Evaluation & Market Comparison

evaluation/calibrate.py

- brier_score() - Probabilistic accuracy metric.
- evaluate_predictions() - Prints Brier + log-loss.
- reliability_diagram() - Saves calibration PNG to outputs/.
- backtest() - Train before cutoff date, test after; refit and score.

evaluation/market.py

- fetch_odds() - The Odds API (requires ODDS_API_KEY env var).
- odds_to_prob() - Decimal odds → vig-stripped probabilities.
- find_edges() - Flags outcomes where |model - market| > threshold (default 5%).
- kelly_fraction() - Position sizing from edge.
- compare_outright_probs() - Tournament winner market vs model.

---

12. Configuration Reference (config.py)

Parameter	Default	Meaning
DECAY_RATE	0.002	Time decay per day (~1 year half-life)
N_SIMULATIONS	100,000	Monte Carlo runs
MIN_DATE	2018-01-01	Earliest match included
ELO_PRIOR_STRENGTH	25.0	Pull toward Elo (0 = off)
L2_REG	0.05	Ridge penalty without Elo
ATTACK_BOUNDS	(-1.5, 1.5)	Per-team attack limits
TOURNAMENT_WEIGHTS	dict	Competition multipliers
GROUPS_2026	dict	12 groups for simulation
ROUND_OF_32	list	Knockout bracket pairings
HOST_COUNTRIES	USA, Canada, Mexico	Home advantage in groups

Parameter	Default	Meaning
EDGE_THRESHOLD	0.05	Market edge flag (5%)
ODDS_API_KEY	env var	Bookmaker API key

---

13. File-by-File Reference

Root level

File	Purpose
app.py	Streamlit interactive dashboard
requirements.txt	Python dependencies
README.md	Quick start and deploy instructions
.streamlit/config.toml	Dashboard theme

world_cup_model/

File	Purpose
__init__.py	Package version
config.py	All hyperparameters and 2026 draw
main.py	CLI entry point: load → fit → simulate → backtest

world_cup_model/data/

File	Purpose
fetch.py	Load results CSV, synthetic data, optional API
clean.py	Filter, names, decay, tournament weights
fetch_elo.py	Download and parse eloratings.net TSVs

world_cup_model/features/

File	Purpose
ratings.py	Elo lookup, merge, prior conversion

world_cup_model/model/

File	Purpose
dixon_coles.py	Core statistical model

world_cup_model/simulation/

File	Purpose
tournament.py	Vectorized World Cup Monte Carlo

world_cup_model/evaluation/

File	Purpose
calibrate.py	Brier, log-loss, backtest, reliability
market.py	Odds API, edges, Kelly

world_cup_model/data_files/ (data, not code)

File	Purpose
results.csv	Historical international matches (you provide)
elo_ratings.csv	Elo history from fetch_elo

world_cup_model/outputs/ (generated)

File	Purpose
win_probs.json	Latest simulation probabilities
reliability_*.png	Calibration charts

scripts/ (utilities)

File	Purpose
smoke_internal.py	End-to-end sanity tests
analyze_ratings.py	Diagnostic script for team strengths
generate_pdf_guide.py	Builds this PDF from markdown

---

14. Outputs & Artifacts

win_probs.json structure:

```
{ "win_probs": { "Spain": 0.12, "Argentina": 0.06, ... }, "finalist_probs": { ... }, "semi_probs": {
... }, "qf_probs": { ... }, "r16_probs": { ... }, "group_advance": { ... } }
```

Probabilities sum to ~1.0 across all 48 teams for win_probs (each team wins some fraction of simulations).

---

15. How to Run the Engine

```
pip install -r requirements.txt # Place results.csv in world_cup_model/data_files/ # Download Elo (optional, ~45 seconds) python -m world_cup_model.data.fetch_elo # Full run python -m world_cup_model.main --n-sims 100000 # Faster test python -m world_cup_model.main --n-sims 5000 --skip-backtest # Tune Elo influence python -m world_cup_model.main --elo-strength 10
```

---

16. Streamlit Web Dashboard

```
streamlit run app.py
```

Tabs:

1. Tournament outlook - Bar charts and tables of win/finalist/advance probabilities

2. Match predictor - Head-to-head win/draw/loss for any two teams

3. Group draw - 2026 groups with advancement %

4. About - Methodology summary

Deploy online: GitHub + share.streamlit.io → main file app.py.

---

17. Known Limitations & Design Choices

1. Independence of matches - Model does not track injuries, squad changes, or within-tournament momentum.

2. Same parameters all tournament - Attack/defense fixed at start; no live updating after each match.

3. 2026 bracket approximation - ROUND_OF_32 in config may need updating when FIFA publishes final pairings.

4. Opponent quality - Dixon-Coles does not explicitly downweight wins vs weak teams (partially addressed by tournament weights + Elo).

5. Team name alignment - Draw teams must match standardized names in results.csv.

6. Market module - Requires live API key; World Cup markets may be empty far from tournament.

7. Not betting advice - Research prototype for learning and demonstration.

---

18. Further Reading

• Dixon, M. J., & Coles, S. G. (1997). Modelling Association Football Scores and Intensities. *Journal of the Royal Statistical Society: Series C.*

• eloratings.net - World Football Elo Ratings methodology

• Mart Jurisoo - International football results dataset (Kaggle)

• The Odds API documentation - Bookmaker odds integration

End of document